

Code Institute Python Linter

Screenshots

app.py



CI Python Linter

```
1 import streamlit as st
2
3 st.set_page_config(
4     page_title="Golf Predictive Analytics Dashboard",
5     page_icon="🏌️",
6     layout="wide"
7 )
8
9 st.title("🏌️ Golf Predictive Analytics Dashboard")
10 st.write("Welcome! Use the sidebar to navigate through the project pages.")
11
```

Settings:



Results:

All clear, no errors found

project_summary.py



CI Python Linter

```
99     "possible. Usually, around 120 players take part in a tournament, so "  
100     "finishing in the top ten would be considered a good achievement for "  
101     "most.</p>"  
102     "<p><b>The Cut:</b> After the first two rounds (36 holes) of an event, "  
103     "a score threshold known as the cut is applied. Only players whose total "  
104     "strokes are at or below this threshold continue to play the final two "  
105     "rounds. The rest are eliminated. The cut ensures that only the top-"  
106     "performing players compete in the weekend rounds.</p>"  
107     "<p><b>Strokes Gained:</b> A performance metric developed to quantify how "  
108     "a golfer's performance compares to the field average. It measures how "  
109     "many strokes a player gains or loses relative to the typical performance "  
110     "of other players on each shot or play category.</p>"  
111     "</div>",  
112     unsafe_allow_html=True  
113 )  
114
```

Settings:



Results:

All clear, no errors found

Player_performance_analysis.py



CI Python Linter

```
1 import streamlit as st
2 import pandas as pd
3 import seaborn as sns
4 import matplotlib.pyplot as plt
5 from src.data_management import load_golf_data, load_player_comparisons
6
7
8 sns.set_style("whitegrid")
9
10
11 # --- Main page ---
12 def page_player_performance_study_body():
13     st.title("🏌️ Player Performance Study")
14
15     df = load_golf_data()
```

Settings:

🌙 ☒ ☀️

Results:

All clear, no errors found

Clustering_study.py



CI Python Linter

```
18 cluster_pipeline = joblib.load(  
19     "outputs/pipelines/clustering_pipeline.pkl"  
20 )  
21  
22 # Toggle summary table  
23 if st.checkbox("Show cluster summary table"):  
24     st.dataframe(cluster_summary.style.format("{:.3f}"))  
25  
26 # Summaries of each cluster  
27 st.markdown(  
28     "<div style='background-color:#e8f4f8;padding:15px;"  
29     "border-radius:10px;margin-top:20px;margin-bottom:10px'"  
30     "<h2>Summarising Each Cluster</h2>"  
31     "<ul>"  
32     "<li><strong>Cluster 0:</strong> Strong chippers, weak driving; "  
33     "steady on the green.</li>"  
34     "<li><strong>Cluster 1:</strong> Weak approach a large issue; "
```

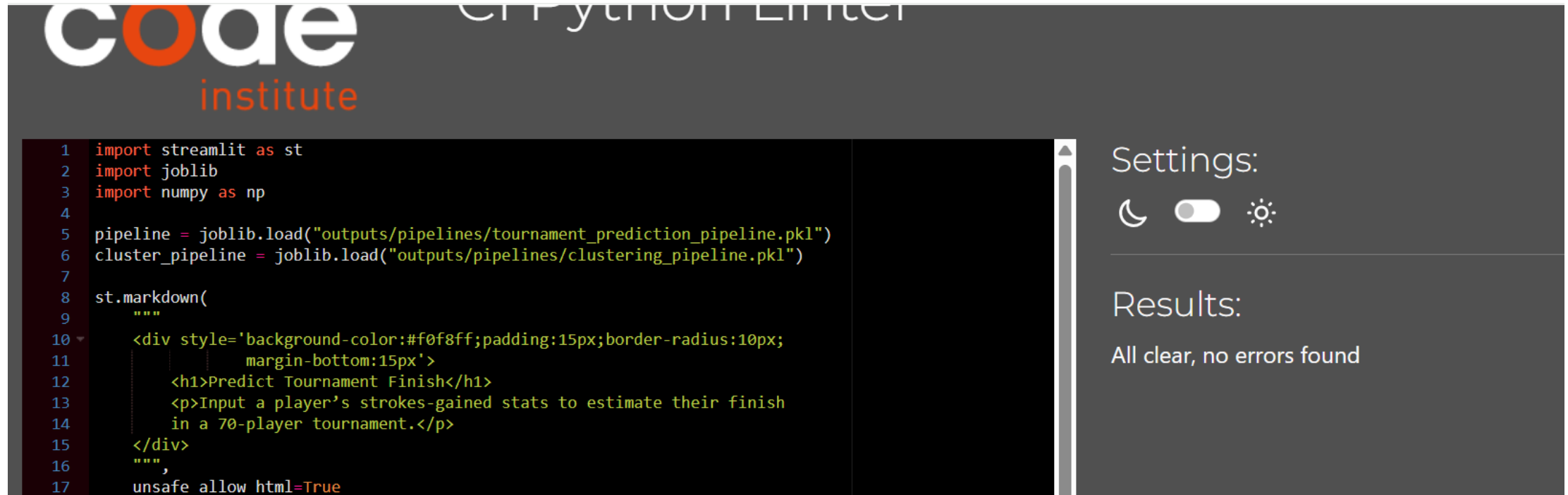
Settings:



Results:

All clear, no errors found

Player_performance_prediction.py



The screenshot displays the Code Institute Python Editor interface. The editor window shows a Python script for a Streamlit application. The code imports streamlit, joblib, and numpy, then loads two pre-trained pipelines. It uses st.markdown to render HTML for a tournament prediction interface. The settings panel on the right shows the application is running in light mode with no errors.

```
1 import streamlit as st
2 import joblib
3 import numpy as np
4
5 pipeline = joblib.load("outputs/pipelines/tournament_prediction_pipeline.pkl")
6 cluster_pipeline = joblib.load("outputs/pipelines/clustering_pipeline.pkl")
7
8 st.markdown(
9     """
10     <div style='background-color:#f0f8ff;padding:15px;border-radius:10px;
11         margin-bottom:15px'>
12         <h1>Predict Tournament Finish</h1>
13         <p>Input a player's strokes-gained stats to estimate their finish
14         in a 70-player tournament.</p>
15     </div>
16     """,
17     unsafe_allow_html=True
```

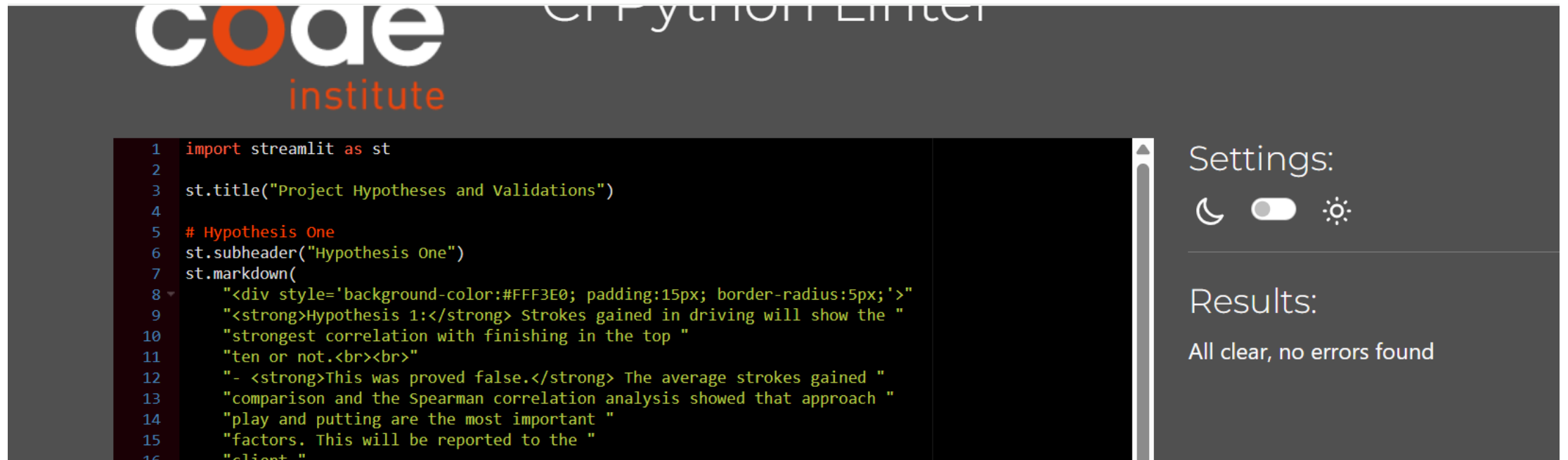
Settings:

☾ ☒ ☀

Results:

All clear, no errors found

Project_hypotheses_and_validation.py



The screenshot shows a web-based code editor interface. At the top left is the 'code institute' logo. The main area is a dark-themed code editor with a Python script. To the right of the editor is a settings panel with a moon icon, a toggle switch, and a sun icon. Below the settings panel is a 'Results:' section displaying the text 'All clear, no errors found'.

```
1 import streamlit as st
2
3 st.title("Project Hypotheses and Validations")
4
5 # Hypothesis One
6 st.subheader("Hypothesis One")
7 st.markdown(
8     "<div style='background-color:#FFF3E0; padding:15px; border-radius:5px;'"
9     "<strong>Hypothesis 1:</strong> Strokes gained in driving will show the "
10     "strongest correlation with finishing in the top "
11     "ten or not.<br><br>"
12     "- <strong>This was proved false.</strong> The average strokes gained "
13     "comparison and the Spearman correlation analysis showed that approach "
14     "play and putting are the most important "
15     "factors. This will be reported to the "
16     "client."
```

Settings:

Results:

All clear, no errors found

ML_cluster_analysis.py



CI Python Linter

```
1 import streamlit as st
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import joblib
5
6
7 def page_cluster_analysis_body():
8
9     # Load clustering pipeline and outputs
10    cluster_pipe = joblib.load(
11        "outputs/pipelines/clustering_pipeline.pkl"
12    )
13    cluster_summary = pd.read_csv(
14        "outputs/cluster/cluster.csv"
15    )
16    silhouette_plot = plt.imread(
17        "outputs/cluster/silhouette_plot.png"
18    )
19
20    # Page header
21    st.write("## ML - Cluster Analysis")
22    st.info(
23        "This analysis groups players based on similarities in their "
```

Settings:

 ☒ 

Results:

All clear, no errors found

ML_performance_prediction_analysis.py



CI Python Linter

```
1 import streamlit as st
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import joblib
5 import json
6 import os
7 import numpy as np
8 from sklearn.metrics import mean_absolute_error
9 from sklearn.metrics import mean_squared_error, r2_score
10
11
12 def page_predict_tournament_body():
13     save_dir = "outputs/pipelines"
14
15     pipe_path = os.path.join(save_dir, "tournament_prediction_pipeline.pkl")
16     best_pipe = joblib.load(pipe_path)
17
18     meta_path = os.path.join(save_dir, "metadata.json")
19     with open(meta_path, "r") as f:
```

Settings:



Results:

All clear, no errors found

Data_management.py



CI Python Linter

```
1 import streamlit as st
2 import pandas as pd
3
4
5 @st.cache_data
6 def load_golf_data(path="outputs/data/final/cleaned_golfdata.csv"):
7     """Load the cleaned golf dataset."""
8     return pd.read_csv(path)
9
10
11 @st.cache_data
12 def load_player_comparisons(
13     top_path="outputs/data/interim/comparison/top_15_players_comparison.csv",
14     mid_path="outputs/data/interim/comparison/mid_range_players_comparison.csv"
15 ):
16     """Load Top-15 and Mid-range precomputed comparison datasets."""
17     top_15 = pd.read_csv(top_path)
18     mid_15 = pd.read_csv(mid_path)
19     return top_15, mid_15
20
```

Settings:



Results:

All clear, no errors found